



山东大学

信息科学与工程学院

2024 – 2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 4

汇编程序编写和提高

专 业 班 级 通信工程 二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2025 年 4 月 7 日

实验报告

【实验目的】

1. 通过实验，完成汇编程序的编写和提高

【实验要求】

1. 编写一个汇编程序，实现字符串的复制功能,并且将复制的字符串显示出来。
2. 利用中断调用，在屏幕上显示 1~9 之间随机数，中断号 86H，且用上课讲的两种方法实现。
3. 键盘输入 10 个学生的成绩，编写一个程序统计 60-69 分，70-79 分，80-89 分，90-99 分及 100 分的人数，分别存放在 Score6，Score7，Score8，Score9 和 Score10 单元中。
输入 10 个学生的成绩如下：

65 98 78 82 88 95 72 62 90 100

【实验具体内容】

【第一个实验】

(1) 实验流程图

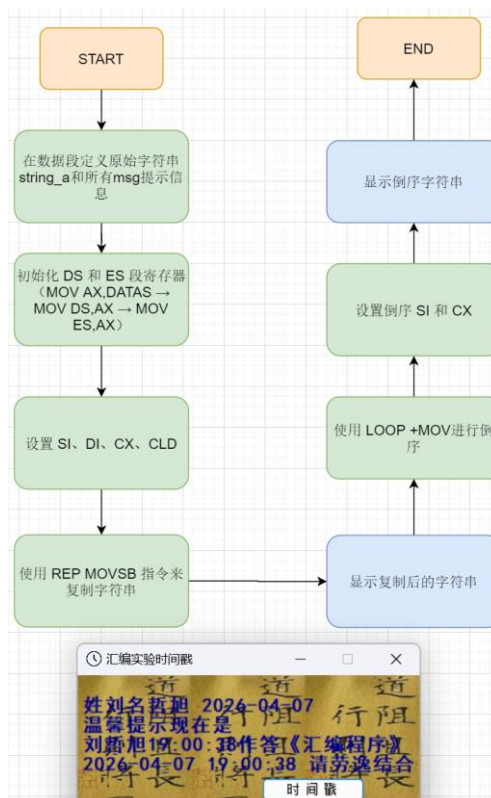


图 1-1 流程图

(2) 实验源代码（粘贴源代码）：

DATAS SEGMENT ; 定义数据段DATAS (存放所有字符串和提示信息)

string_a db 'The School of Information Science and Engineering Shandong University','\$' ; 定义源字符串string_a, 以'\$'作为DOS显示结束符

string_b db 100 dup(?) ; 定义目标字符串string_b, 预留100字节空间, 用于存放复制结果

msg_original db 'string original:\$' ; 定义“string original:”标签, 用于显示原始字符串

msg_copied db 0Dh,0Ah,'copied:\$' ; 定义“copied:”标签, 前面加0Dh,0Ah实现换行

msg_reversed db 0Dh,0Ah,'reversed:\$' ; 定义“reversed:”标签, 前面加0Dh,0Ah实现换行

msg_press db 0Dh,0Ah,0Dh,0Ah,'Press any key to continue\$' ; 定义“Press any key to continue”提示信息, 前面加两个换行

DATAS ENDS ; 数据段DATAS定义结束

STACKS SEGMENT ; 定义堆栈段STACKS (题目要求必须编写堆栈段代码)

db 200 dup(0) ; 分配200字节的堆栈空间, 用于存放临时数据和返回地址

STACKS ENDS ; 堆栈段STACKS定义结束

CODES SEGMENT ; 定义代码段CODES

ASSUME CS:CODES, DS:DATAS, SS:STACKS, ES:DATAS ; 告诉汇编器: CS

指向代码段、DS/ES都指向DATAS段

START: ; 程序入口标号（程序从这里开始执行）

行)

MOV AX, DATAS ; 把DATAS段的段地址装入AX寄存器

寄存器

MOV DS, AX ; 将AX中的段地址赋给DS，使DS指向数据段

指向数据段

MOV ES, AX ; 将AX中的段地址赋给ES，使ES也指向同一数据段

指向同一数据段

; 显示 string original:

LEA DX, msg_original ; DX指向“string original:”标签的偏移地址

移地址

MOV AH, 9 ; AH=9, 表示调用DOS 9号功能(显示以\$结尾的字符串)

示以\$结尾的字符串)

INT 21H ; 调用DOS中断21H，在屏幕显示“string original:”

“string original:”

LEA DX, string_a ; DX指向源字符串string_a的偏移地址

MOV AH, 9 ; AH=9, 准备显示原字符串

INT 21H ; 调用DOS中断21H，显示原始字符串

串

; 使用 REP MOVSB 复制字符串

LEA SI, string_a ; SI指向源字符串string_a的偏移地址

```

    LEA DI, string_b                ; DI指向目标字符串string_b的偏移地
址
    MOV CX, 70                      ; CX=70 (字符串实际长度69个字符
+ 1个'$'结束符)
    CLD                            ; 清方向标志DF=0, 使字符串操作
正向进行
    REP MOVSB                      ; 重复执行MOVSB CX次, 完成字
符串从string_a到string_b的复制
    ; 显示 copied:
    LEA DX, msg_copied             ; DX指向“copied:”标签的偏移地址
    MOV AH, 9                     ; AH=9, 准备显示copied标签
    INT 21H                       ; 调用DOS中断21H, 显示“copied:”
并换行
    LEA DX, string_b              ; DX指向复制后的字符串string_b的
偏移地址
    MOV AH, 9                     ; AH=9, 准备显示复制结果
    INT 21H                       ; 调用DOS中断21H, 显示复制后的
字符串
    ; 显示 reversed:
    LEA DX, msg_reversed          ; DX指向“reversed:”标签的偏移地
址
    MOV AH, 9                     ; AH=9, 准备显示reversed标签

```

```

INT 21H                                ; 调用DOS中断21H, 显示
“reversed:”并换行

; 倒序显示字符串 (选做部分实现)

MOV SI, OFFSET string_b + 68          ; SI指向string_b的最后一个有效字
符

MOV CX, 69                            ; CX=69, 要倒序打印的字符个数

REVERSE_LOOP:                          ; 倒序循环标号

MOV DL, [SI]                           ; 把当前字符从[SI]取出放入DL

MOV AH, 2                              ; AH=2, 表示调用DOS 2号功能

INT 21H                                ; 调用DOS中断21H, 屏幕显示当前
字符

DEC SI                                 ; SI减1, 指向前一个字符

LOOP REVERSE_LOOP                      ; CX减1, 若CX不为0则继续循环

; 显示按任意键继续提示

LEA DX, msg_press                      ; DX指向“Press any key to
continue”提示信息的偏移地址

MOV AH, 9                              ; AH=9, 准备显示提示信息

INT 21H                                ; 调用DOS中断21H, 显示按键提示

; 等待用户按任意键

MOV AH, 1                              ; AH=1, 表示调用DOS 1号功能

INT 21H                                ; 调用DOS中断21H, 程序暂停, 直
到用户按下任意键

```

```

MOV AH, 4CH                ; AH=4CH, 表示程序正常结束并
                             返回DOS

INT 21H                    ; 调用DOS中断21H, 结束程序并返
                             回操作系统

CODES ENDS                ; 代码段CODES定义结束

END START                  ; 整个程序结束, 告诉汇编器从
                             START标号开始执行
    
```

(3) 实验过程

①打开 Masm fow Windows 集成实验环境 2015 后将代码输入，将文件保存为 COPYANDREVERSE.asm，点击运行，如图 1-2 所示：

```

01 DATAS SEGMENT                ; 定义数据段DATAS (存放所有字符串和提示信息)
02     string_a      db 'The School of Information Science and Engineering Shandong University','$' ; 定义目标字符串string_b, 预留100字节空间, 用于存放复制结果
03     string_b      db 100 dup(?)
04     msg_original   db 'string original:$' ; 定义 "string original:" 标签, 用于显示原始字符串
05     msg_copied     db 0Dh,0Ah,'copied:$' ; 定义 "copied:" 标签, 前面加0Dh,0Ah实现换行
06     msg_reversed   db 0Dh,0Ah,'reversed:$' ; 定义 "reversed:" 标签, 前面加0Dh,0Ah实现换行
07     msg_press      db 0Dh,0Ah,0Dh,0Ah,'Press any key to continue$' ; 定义 "Press any key to continue"
08 DATAS ENDS                ; 数据段DATAS定义结束
09
10 STACKS SEGMENT              ; 定义堆栈段STACKS (题目要求必须编写堆栈段代码)
11     db 200 dup(0)          ; 分配200字节的堆栈空间, 用于存放临时数据和返回地址
12 STACKS ENDS                ; 堆栈段STACKS定义结束
13
14 CODES SEGMENT                ; 定义代码段CODES
15     ASSUME CS:CODES, DS:DATAS, SS:STACKS, ES:DATAS ; 告诉汇编器: CS指向代码段、DS/ES都指向DATAS段
16
17 START:                      ; 程序入口标号 (程序从这里开始执行)
18     MOV AX, DATAS            ; 把DATAS段的段地址装入AX寄存器
19     MOV DS, AX               ; 将AX中的段地址赋给DS, 使DS指向数据段
20     MOV ES, AX               ; 将AX中的段地址赋给ES, 使ES也指向同一数据段
21
22     ; 显示 string original:
23     LEA DX, msg_original      ; DX指向 "string original:" 标签的偏移地址
24     MOV AH, 9                 ; AH=9, 表示调用DOS 9号功能 (显示以$结尾的字符串)
25     INT 21H                  ; 调用DOS中断21H, 在屏幕显示 "string original:"
26     LEA DX, string_a         ; DX指向源字符串string_a的偏移地址
27     MOV AH, 9                 ; AH=9, 准备显示原字符串
28     INT 21H                  ; 调用DOS中断21H, 显示原始字符串
29
30     ; 使用 REP MOVSB 复制字符串
31     LEA SI, string_a          ; SI指向源字符串string_a的偏移地址
32     LEA DI, string_b          ; DI指向目标字符串string_b的偏移地址
33     MOV CX, 70                ; 复制70个字节
34     CLD                      ; 清除方向标志, 使DI和SI都递增
35     REP MOVSB                 ; 重复执行MOVSB指令, 复制字符串
36
37     ; 显示 copied:
38     LEA DX, msg_copied        ; DX指向 "copied:" 标签的偏移地址
39     MOV AH, 9                 ; AH=9, 表示调用DOS 9号功能 (显示以$结尾的字符串)
40     INT 21H                  ; 调用DOS中断21H, 在屏幕显示 "copied:"
41     LEA DX, string_b         ; DX指向目标字符串string_b的偏移地址
42     MOV AH, 9                 ; AH=9, 准备显示复制后的字符串
43     INT 21H                  ; 调用DOS中断21H, 显示复制后的字符串
    
```

图 1-2: COPYANDREVERSSE.ASM

②运行结果如下，显示了复制后的字符串和反向处理的字符串，如图 1-3 所示：

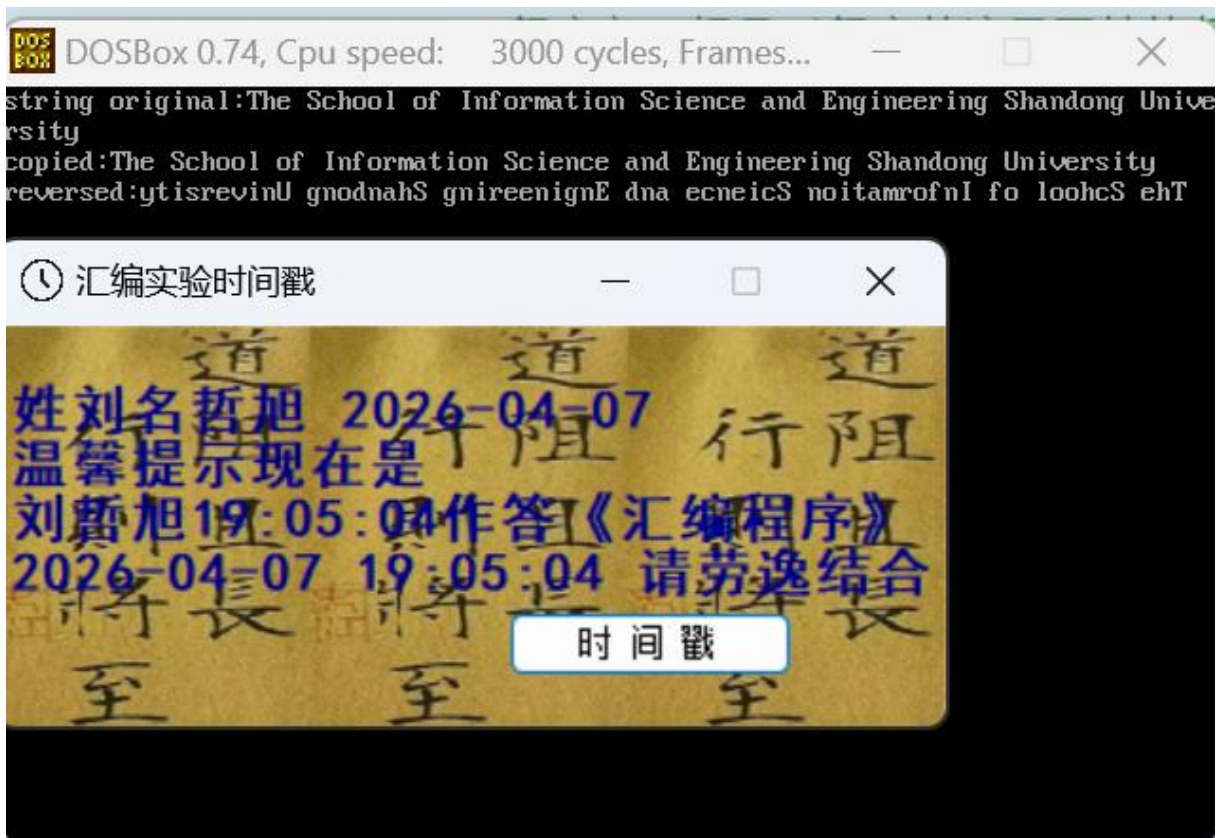


图 1-3：运行显示复制后的和反向处理的字符串

【第二个实验】

(1) 实验流程图：

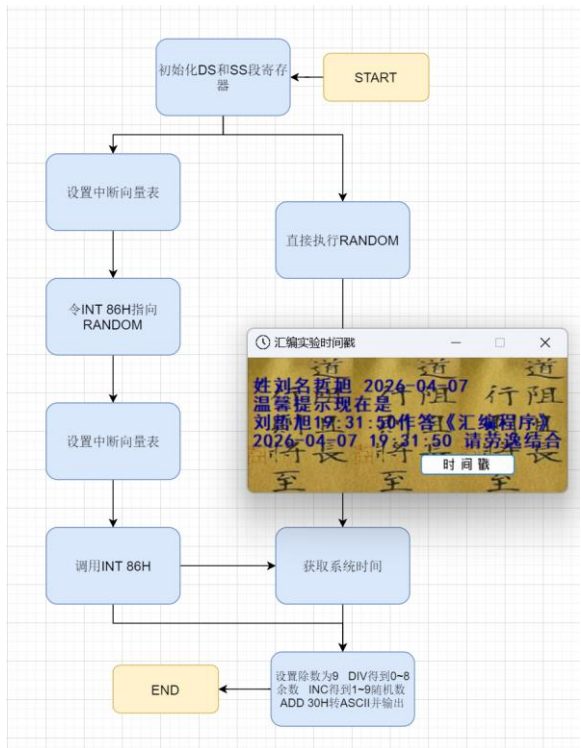


图 2-1：流程图

(2) 实验源代码（粘贴源代码）：

①定义法

```

DATAS SEGMENT                                ; 数据段开始

    MSG DB 'random',13,10,'$'                ; 提示信息告知用户即将显示1到9随机数

DATAS ENDS                                    ; 数据段结束

STACKS SEGMENT stack                          ; 栈段开始

    DW 8 DUP (0)                             ; 分配16字节的栈空间

STACKS ENDS                                  ; 栈段结束

CODES SEGMENT                                ; 代码段开始

    ASSUME CS:CODES, DS:DATAS, SS:STACKS     ; 设置段寄存器的假设

MAIN:                                         ; 程序主入口

    MOV AX, DATAS                            ; 将数据段地址加载到AX寄存器

    MOV DS, AX                               ; 设置DS指向数据段

    MOV AX, STACKS                           ; 将栈段地址加载到AX寄存器

    MOV SS, AX                               ; 设置SS指向栈段

    XOR AX, AX                                ; 清零AX准备访问中断向量表

    MOV ES, AX                               ; ES指向中断向量表所在段

    MOV BX, 0218H                             ; 计算INT 86H的中断向量偏移地址

    LEA AX, RANDOM                           ; 获取中断服务程序偏移地址

    MOV ES:[BX], AX                           ; 将偏移地址写入向量表
    
```

```
MOV AX, CS           ; 获取当前代码段地址
MOV ES:[BX+2], AX    ; 将段地址写入向量表完成安装
MOV AH, 09H          ; 设置AH为09H准备显示提示字
```

字符串

```
LEA DX, MSG          ; DX指向提示信息地址
INT 21H              ; 调用DOS中断显示提示信息
INT 86H              ; 调用自定义中断86H执行随机数显
```

示

```
MOV AH, 4CH          ; 设置AH为4CH准备结束程序
INT 21H              ; 调用DOS中断退出程序
```

RANDOM: ; 中断服务程序标号

```
MOV AH, 2CH          ; 设置AH为2CH准备获取系统时
```

间

```
INT 21H              ; 调用DOS中断获取当前时间
MOV AL, DL           ; 取毫秒值作为随机种子
MOV AH, 0            ; 清零AH准备除法
MOV BL, 9            ; 设置除数为9
DIV BL               ; 除以9得到0到8的余数
INC AH               ; 余数加1得到1到9的随机数
ADD AH, 30H          ; 转换为ASCII字符
MOV DL, AH           ; DL存放要显示的字符
MOV AH, 2            ; 设置AH为2准备显示单个字符
```

```

INT 21H                                ; 调用DOS中断显示随机数

IRET                                    ; 中断返回

CODES ENDS                             ; 代码段结束

END MAIN                               ; 指定程序入口点

②DOS INT21H 法

DATAS SEGMENT                          ; 数据段开始

    MSG DB 'random',13,10,'$'          ; 提示信息告知用户即将显示1到9随
机数

DATAS ENDS                             ; 数据段结束

STACKS SEGMENT stack                  ; 栈段开始

    DW 8 DUP (0)                      ; 分配16字节的栈空间

STACKS ENDS                           ; 栈段结束

CODES SEGMENT                          ; 代码段开始

    ASSUME CS:CODES, DS:DATAS, SS:STACKS ; 设置段寄存器的假设

MAIN:                                  ; 程序主入口

    MOV AX, DATAS                      ; 将数据段地址加载到AX寄存器

    MOV DS, AX                         ; 设置DS指向数据段

    MOV AX, STACKS                     ; 将栈段地址加载到AX寄存器

    MOV SS, AX                         ; 设置SS指向栈段

    MOV AH, 09H                        ; 设置AH为09H准备显示提示字
符串

    LEA DX, MSG                        ; DX指向提示信息地址

```

```

INT 21H                                ; 调用DOS中断显示提示信息

CALL RANDOM                            ; 调用RANDOM子程序生成并
显示随机数

MOV AH, 4CH                            ; 设置AH为4CH准备结束程序

INT 21H                                ; 调用DOS中断退出程序

RANDOM:                                 ; 随机数生成子程序标号

MOV AH, 2CH                            ; 设置AH为2CH准备获取系统时
间

INT 21H                                ; 调用DOS中断获取当前时间

MOV AL, DL                             ; 取毫秒值作为随机种子

MOV AH, 0                              ; 清零AH准备除法

MOV BL, 9                              ; 设置除数为9

DIV BL                                 ; 除以9得到0到8的余数

INC AH                                 ; 余数加1得到1到9的随机数

ADD AH, 30H                            ; 转换为ASCII字符

MOV DL, AH                             ; DL存放要显示的字符

MOV AH, 2                              ; 设置AH为2准备显示单个字符

INT 21H                                ; 调用DOS中断显示随机数

RET                                    ; 子程序返回

CODES ENDS                            ; 代码段结束

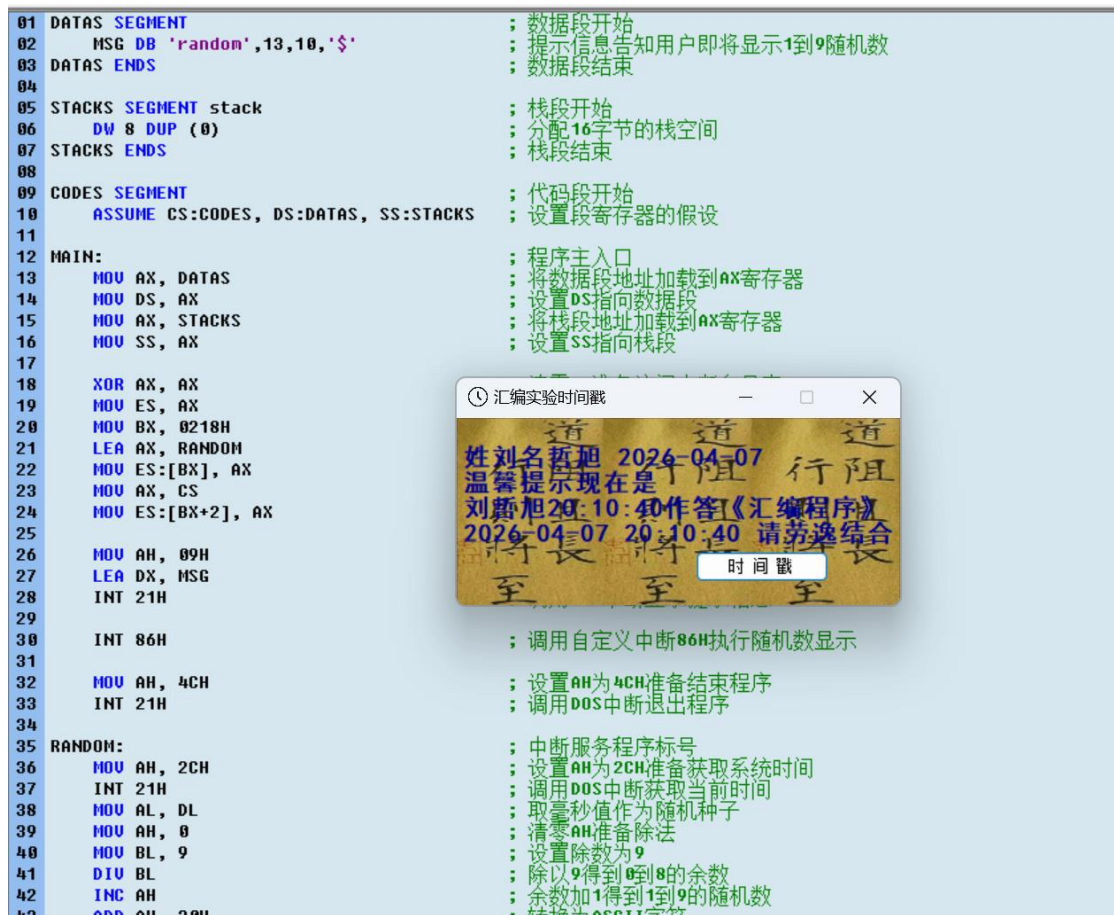
END MAIN                              ; 指定程序入口点

```

(3) 实验过程

①打开 Masm fow Windows 集成实验环境 2015 后将代码输入，保存为 random1.asm，随

点击运行运行。如图 2-2 所示：

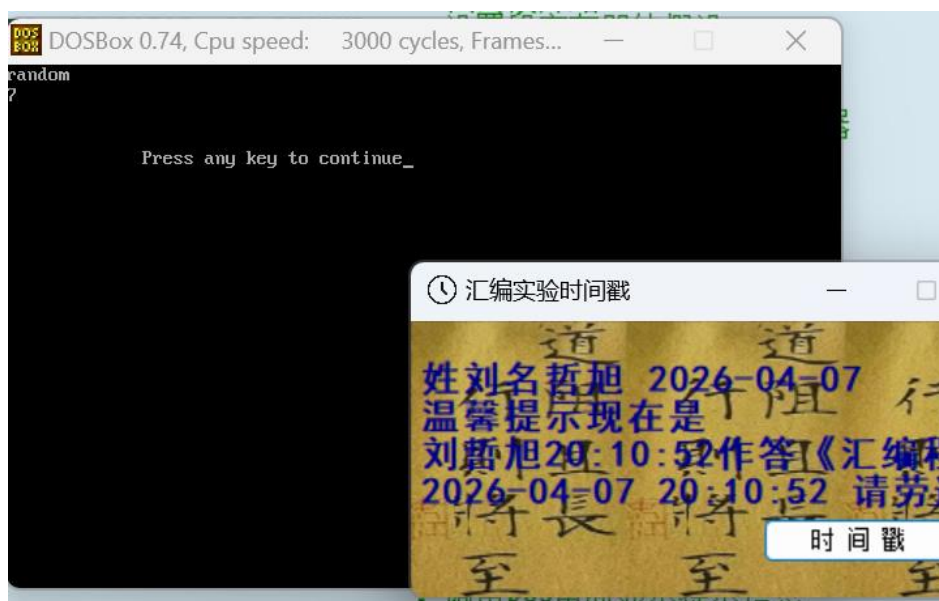


```

01 DATAS SEGMENT                ; 数据段开始
02     MSG DB 'random',13,10,'$' ; 提示信息告知用户即将显示1到9随机数
03 DATAS ENDS                  ; 数据段结束
04
05 STACKS SEGMENT stack        ; 栈段开始
06     DW 8 DUP (0)             ; 分配16字节的栈空间
07 STACKS ENDS                ; 栈段结束
08
09 CODES SEGMENT                ; 代码段开始
10     ASSUME CS:CODES, DS:DATAS, SS:STACKS ; 设置段寄存器的假设
11
12 MAIN:                        ; 程序主入口
13     MOV AX, DATAS             ; 将数据段地址加载到AX寄存器
14     MOV DS, AX               ; 设置DS指向数据段
15     MOV AX, STACKS           ; 将栈段地址加载到AX寄存器
16     MOV SS, AX              ; 设置SS指向栈段
17
18     XOR AX, AX               ; 将AX寄存器清零
19     MOV ES, AX               ; 设置ES指向堆栈段
20     MOV BX, 0218H            ; 设置BX寄存器为0218H
21     LEA AX, RANDOM           ; 将RANDOM地址加载到AX寄存器
22     MOV ES:[BX], AX          ; 将AX寄存器内容加载到ES:[BX]地址
23     MOV AX, CS               ; 将CS寄存器内容加载到AX寄存器
24     MOV ES:[BX+2], AX        ; 将AX寄存器内容加载到ES:[BX+2]地址
25
26     MOV AH, 09H              ; 设置AH寄存器为09H
27     LEA DX, MSG              ; 将MSG地址加载到DX寄存器
28     INT 21H                  ; 调用DOS中断21H
29
30     INT 86H                  ; 调用自定义中断86H执行随机数显示
31
32     MOV AH, 4CH              ; 设置AH寄存器为4CH
33     INT 21H                  ; 调用DOS中断21H
34
35 RANDOM:                      ; 中断服务程序标号
36     MOV AH, 2CH              ; 设置AH寄存器为2CH
37     INT 21H                  ; 调用DOS中断21H
38     MOV AL, DL               ; 将DL寄存器内容加载到AL寄存器
39     MOV AH, 0                ; 将AH寄存器清零
40     MOV BL, 9                ; 设置BL寄存器为9
41     DIV BL                   ; 将AL寄存器内容除以BL寄存器
42     INC AH                   ; 将AH寄存器内容加1
43     ADD AH, 30H              ; 将AH寄存器内容加上30H
    
```

图 2-2: 定义法 random1.axm

②运行生成随机数如如 2-3 所示：



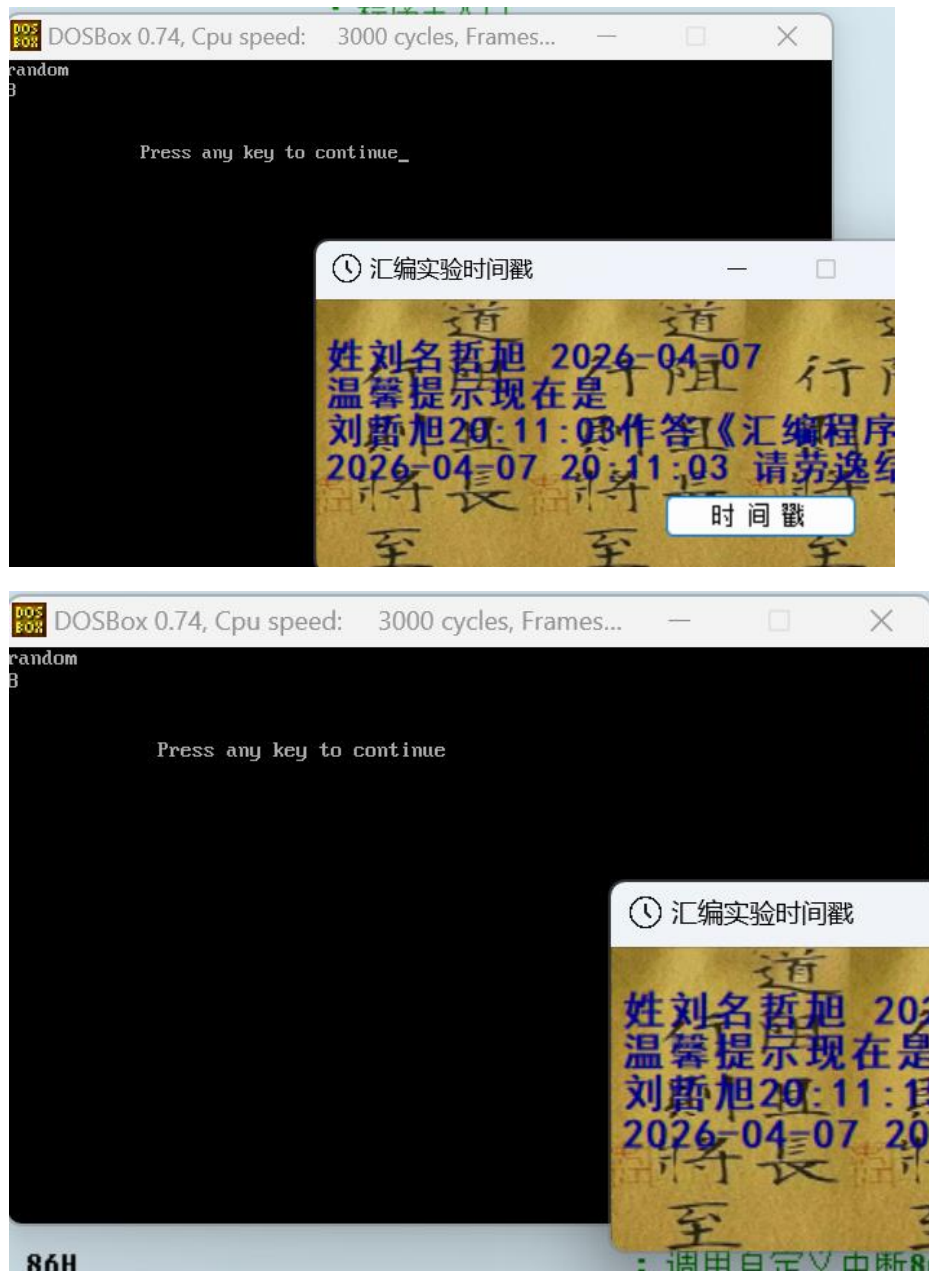


图 2-3：定义法生成随机数

③打开 Masm fow Windows 集成实验环境 2015 后将代码输入，保存为 random1.asm，随点击运行运行。如图 2-4 所示：

```
01 DATAS SEGMENT ; 数据段开始
02     MSG DB 'random',13,10,'$' ; 提示信息告知用户即将显示1到9随机数
03 DATAS ENDS ; 数据段结束
04
05 STACKS SEGMENT stack ; 栈段开始
06     DW 8 DUP (0) ; 分配16字节的栈空间
07 STACKS ENDS ; 栈段结束
08
09 CODES SEGMENT ; 代码段开始
10     ASSUME CS:CODES, DS:DATAS, SS:STACKS ; 设置段寄存器的假设
11
12 MAIN: ; 程序主入口
13     MOV AX, DATAS ; 将数据段地址加载到AX寄存器
14     MOV DS, AX ; 设置DS指向数据段
15     MOV AX, STACKS ; 将栈段地址加载到AX寄存器
16     MOV SS, AX ; 设置SS指向栈段
17
18     MOV AH, 09H ; 设置AH为09H准备显示提示字符串
19     LEA DX, MSG ; 将提示信息地址加载到DX寄存器
20     INT 21H
21
22     CALL RANDOM
23
24     MOV AH, 4CH ; 设置AH为4CH准备返回DOS
25     INT 21H
26
27 RANDOM:
28     MOV AH, 2CH ; 设置AH为2CH准备取毫秒值
29     INT 21H ; 取毫秒值作为随机种子
30     MOV AL, DL ; 将DL中的值移动到AL
31     MOV AH, 0 ; 清零AH准备除法
32     MOV BL, 9 ; 设置除数为9
33     DIV BL ; 除以9得到0到8的余数
34     INC AH ; 余数加1得到1到9的随机数
35     ADD AH, 30H ; 转换为ASCII字符
36     MOV DL, AH ; DL存放要显示的字符
37     MOV AH, 2 ; 设置AH为2准备显示单个字符
38     INT 21H ; 调用DOS中断显示随机数
39     RET ; 子程序返回
40
41 CODES ENDS ; 代码段结束
42 END MAIN ; 指定程序入口点
```

图 2-4: DOS INT21H 法 random2.asm





图 2-5: DOS INT21H 法生成随机数

【第三个实验】

(1) 实验流程图:

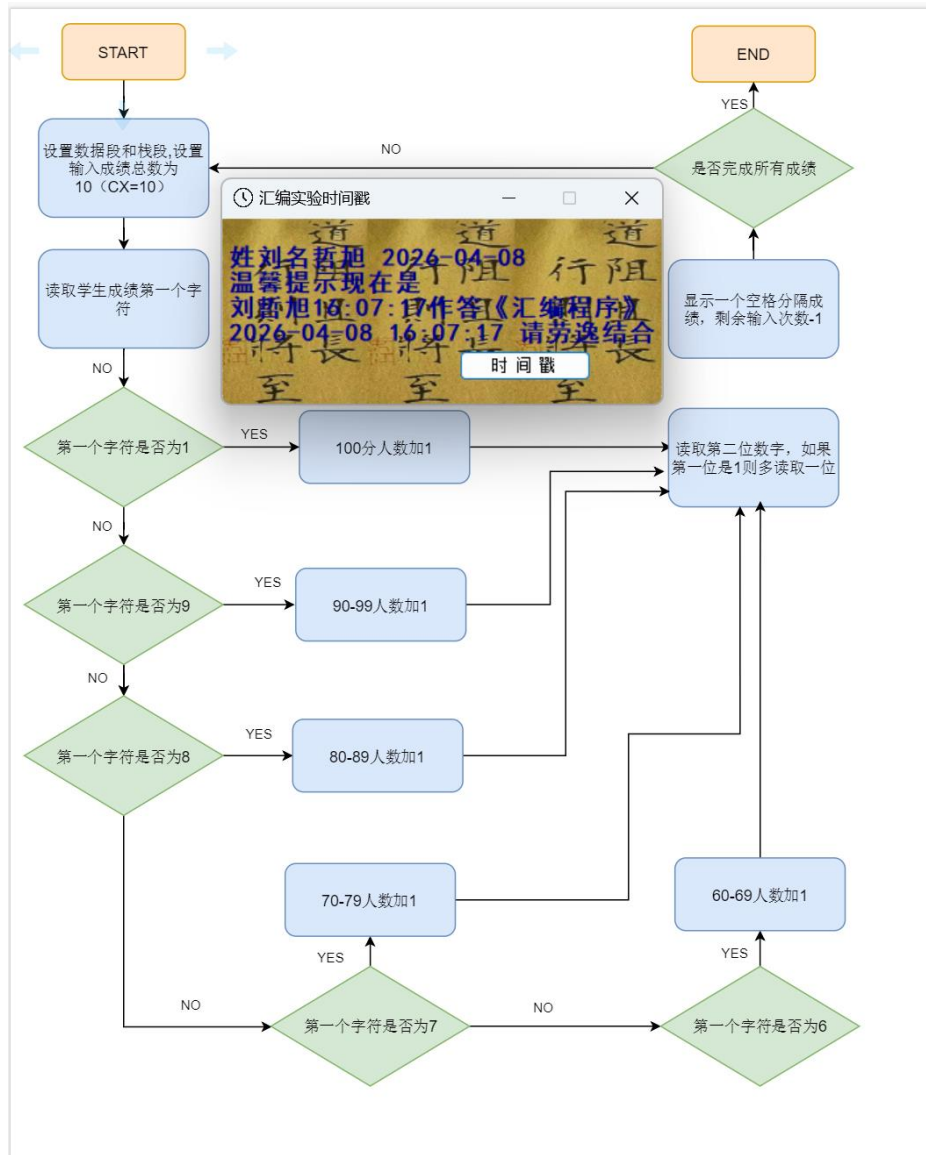


图 3-1: 流程图

(2) 实验源代码:

DATAS SEGMENT

; 数据段开始

MSG_INPUT DB 'scores:',13,10,'\$' ; 提示输入10个学生的成绩

MSG_60 DB 13,10,'60~69: ','\$' ; 提示60~69分的人数

MSG_70 DB 13,10,'70~79: ','\$' ; 提示70~79分的人数

MSG_80 DB 13,10,'80~89: ','\$' ; 提示80~89分的人数

MSG_90 DB 13,10,'90~99: ','\$' ; 提示90~99分的人数

MSG_100 DB 13,10,'100: ','\$' ; 提示100分的人数

；存放各分数段人数（用字节变量，初始为0）

COUNT_60 DB 0 ; 60~69分人数

COUNT_70 DB 0 ; 70~79分人数

COUNT_80 DB 0 ; 80~89分人数

COUNT_90 DB 0 ; 90~99分人数

COUNT_100 DB 0 ; 100分人数

DATAS ENDS

STACKS SEGMENT STACK ; 栈段

DB 16 DUP (?) ; 分配16字节栈空间

STACKS ENDS

CODES SEGMENT ; 代码段开始

ASSUME CS:CODES, DS:DATAS, SS:STACKS ; 段寄存器假设

MAIN: ; 主程序入口

MOV AX, DATAS ; 获取数据段地址

MOV DS, AX ; 设置DS指向数据段

MOV AX, STACKS ; 获取栈段地址

MOV SS, AX ; 设置SS指向栈段

；显示输入提示

MOV AH, 09H ; DOS 09H功能：显示以\$结尾的字符串

LEA DX, MSG_INPUT ; 加载输入提示地址

INT 21H ; 显示 "scores:"

MOV CX, 10 ; 循环10次，输入10个成绩

```

INPUT_LOOP:                                ; 输入循环

    MOV AH, 01H                            ; 01H功能: 键盘读入一个字符 (带回显)

    INT 21H                                ; 读取十位数字到AL

    CALL CLASSIFY                          ; 调用分类子程序判断分数段

    MOV AH, 01H                            ; 再次读取个位数字 (为了让用户正常输入)

    INT 21H

    MOV AH, 02H                            ; 02H功能: 显示单个字符

    MOV DL, ' '                            ; 显示一个空格分隔成绩

    INT 21H

    LOOP INPUT_LOOP                        ; 继续输入直到10个完成

; 显示统计结果

    LEA DX, MSG_60                         ; 显示60~69提示

    MOV AH, 09H

    INT 21H

    CALL DispDigit                        ; 调用显示数字子程序

    MOV DL, COUNT_60                      ; 取出60~69人数

    CALL DispDigit                        ; 显示人数

    LEA DX, MSG_70                         ; 显示70~79提示

    MOV AH, 09H

    INT 21H

    MOV DL, COUNT_70

    CALL DispDigit
    
```

```

LEA DX, MSG_80          ; 显示80~89提示

MOV AH, 09H

INT 21H

MOV DL, COUNT_80

CALL DispDigit

LEA DX, MSG_90          ; 显示90~99提示

MOV AH, 09H

INT 21H

MOV DL, COUNT_90

CALL DispDigit

LEA DX, MSG_100         ; 显示100提示

MOV AH, 09H

INT 21H

MOV DL, COUNT_100

CALL DispDigit

; 程序退出

MOV AH, 4CH             ; 4CH功能: 退出程序

INT 21H

; 子程序

CLASSIFY PROC NEAR      ; 分类子程序 (根据十位数字判断)

    CMP AL, '6'         ; 是否为'6' (60~69分)

    JE SCORE_60
    
```

```

    CMP AL, '7'                ; 是否为'7' (70~79分)
    JE SCORE_70

    CMP AL, '8'                ; 是否为'8' (80~89分)
    JE SCORE_80

    CMP AL, '9'                ; 是否为'9' (90~99分)
    JE SCORE_90

    CMP AL, '1'                ; 是否为'1' (100分)
    JE SCORE_100

    RET                        ; 都不是则直接返回

SCORE_60:

    INC COUNT_60              ; 60~69人数加1
    JMP DONE

SCORE_70:

    INC COUNT_70              ; 70~79人数加1
    JMP DONE

SCORE_80:

    INC COUNT_80              ; 80~89人数加1
    JMP DONE

SCORE_90:

    INC COUNT_90              ; 90~99人数加1
    JMP DONE

SCORE_100:

```

```

INC COUNT_100                ; 100分人数加1

MOV AH, 01H                  ; 100分是三位数，多读一个'0'

INT 21H

JMP DONE

DONE:

RET

CLASSIFY ENDP

; 显示一位数字 (0~9)

DispDigit PROC NEAR

    ADD DL, '0'               ; 把数值0~9转换为ASCII字符 '0'~'9'

    MOV AH, 02H               ; 02H功能：显示单个字符

    INT 21H                   ; 显示数字

    RET

DispDigit ENDP

CODES ENDS                  ; 代码段结束

END MAIN                    ; 程序入口

```

(3) 实验过程:

①打开 Masm fow Windows 集成实验环境 2015 后将代码输入，保存为 score.asm，点击运行。如图 3-2 所示：

```
001 DATAS SEGMENT ; 数据段开始
002 MSG_INPUT DB 'scores:',13,10,'$' ; 提示输入10个学生的成绩
003 MSG_60 DB 13,10,'60~69: ','$' ; 提示60~69分的人数
004 MSG_70 DB 13,10,'70~79: ','$' ; 提示70~79分的人数
005 MSG_80 DB 13,10,'80~89: ','$' ; 提示80~89分的人数
006 MSG_90 DB 13,10,'90~99: ','$' ; 提示90~99分的人数
007 MSG_100 DB 13,10,'100: ','$' ; 提示100分的人数
008
009 ; 存放各分数段人数 (用字节变量, 初始为0)
010 COUNT_60 DB 0 ; 60~69分人数
011 COUNT_70 DB 0 ; 70~79分人数
012 COUNT_80 DB 0 ; 80~89分人数
013 COUNT_90 DB 0 ; 90~99分人数
014 COUNT_100 DB 0 ; 100分人数
015
016 DATAS ENDS
017
018 STACKS SEGMENT STACK ; 栈段
019 DB 16 DUP (?) ; 分配16字节栈空间
020 STACKS ENDS
021
022 CODES SEGMENT ; 代码段开始
023 ASSUME CS:CODES, DS:DATAS, SS:STACKS ; 段寄存器假设
024
025 MAIN: ; 主程序入口
026 MOV AX, DATAS ; 获取数据段地址
027 MOV DS, AX ; 设置DS指向数据段
028
029 MOV AX, STACKS ; 获取栈段地址
030 MOV SS, AX ; 设置SS指向栈段
031
032 ; 显示输入提示
033 MOV AH, 09H ; DOS 09H功能: 显示以$结尾的字符串
034 LEA DX, MSG_INPUT ; 加载输入提示地址
035 INT 21H ; 显示 "scores:"
036
037 MOV CX, 10 ; 循环10次, 输入10个成绩
038
039 INPUT_LOOP: ; 输入循环
040 MOV AH, 01H ; 01H功能: 键盘读入一个字符 (带回显)
041 INT 21H ; 读取十位数字到AL
042 CALL CLASSIFY ; 调用分类子程序判断分数段
043
```

图 3-2: score.asm

②运行结果如下:



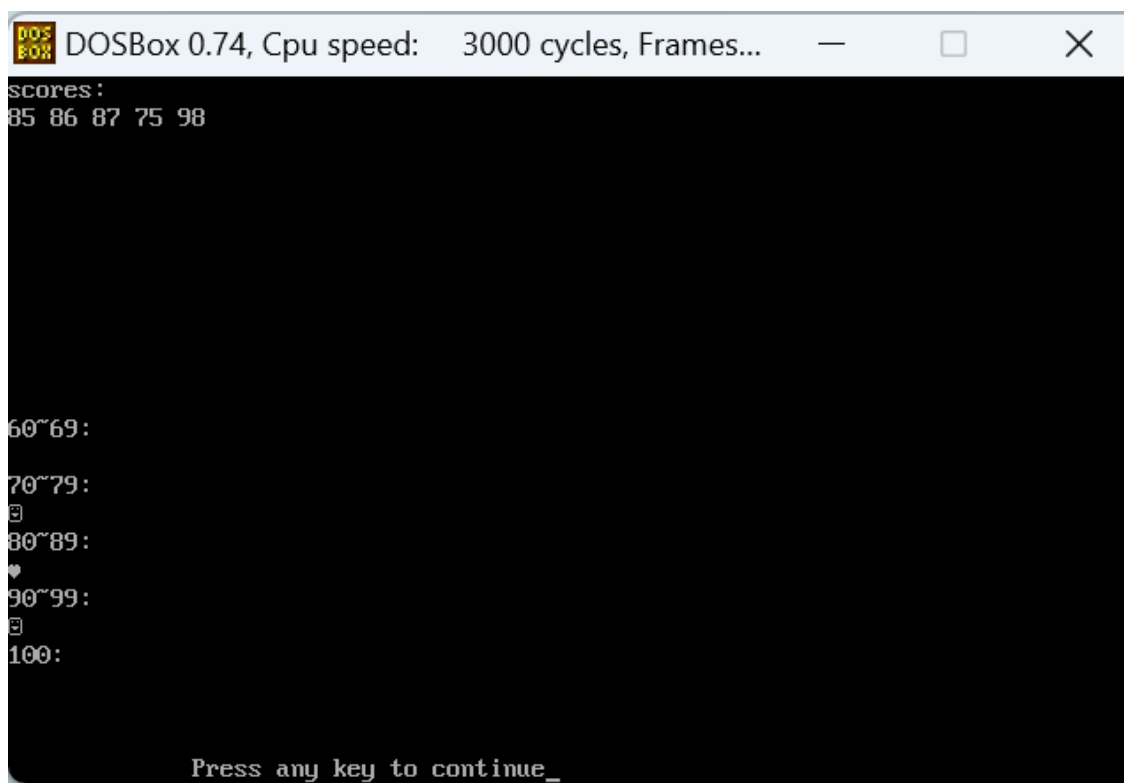
图 3-2: 运行结果

【实验心得】

在本次汇编语言成绩统计程序实验中，我遇到了较多实际问题，也通过反复调试和修改加深了对汇编程序特点的理解。

首先是第三个实验，在编写本程序时，我主要采用了固定循环 10 次、每次读取两个字符的输入方式，并通过 CLASSIFY 子程序根据十位数字进行分数段统计。然而，实际运行中却出现了两个比较棘手的问题。一是当输入少于 10 个成绩时，需要按很多次回车键程序才会显示统计结果；二是统计结果经常显示为乱码或完全错误。经过反复检查，发现前者是由于键盘缓冲区中残留了大量未处理的回车符（0Dh），后者则是因为 COUNT_xx 变量初始定义为字符 '0'，执行 INC 操作后变成了不可打印字符，导致用 INT 21H 显示时出现乱码。

为了解决这些问题，我将计数变量改为数值 DB 0，并专门编写了 DispDigit 子程序负责将数字转换为 ASCII 字符后再显示，从而彻底消除了乱码现象。同时，我也意识到了固定字符读取方式的局限性：在输入逻辑不够灵活的情况下，很容易受到键盘缓冲区的影响。这些调试经历让我深刻体会到，汇编语言对细节的要求远高于高级语言，一个小小的变量类型选择或输入处理不当，就可能引发看似莫名其妙的问题。



```

DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
scores:
85 86 87 75 98

60~69:
70~79:
80~89:
90~99:
100:

Press any key to continue_
    
```

回顾整个实验系列，我最大的感触是：汇编语言的学习虽然充满挑战，但它让我真正理解了计算机底层的工作机制。从高级语言一行简单的 if 语句，到汇编中需要手动管理标志寄存器和跳转指令，这种思维转换的过程虽然艰难，却让我对计算机如何执行程序有了更本质的认识。那些曾经令我困惑的寄存器冲突、段地址计算和输入输出处理，如今都变成了宝贵的经验。

总之，本次实验让我更加清醒地认识到，编程能力的提升并非一蹴而就，而是通过不断发现问题、分析问题和解决问题来逐步实现的。汇编语言的学习不仅传授了具体的指令用法，更重要的是培养了严谨、细致和底层思维的能力，这将对我未来学习其他计算机课程产生长期而积极的影响。



知识点

中断服务子程序（ISR）与普通子程序的区别可以从以下几个方面进行深入分析：

1. **程序关系特性** 中断服务程序与正在运行的主程序之间是异步关系，由外部硬件事件或软件 INT 指令触发，属于独立的执行流程。普通子程序与主程序是同步关系，是主程序主动调用的一个功能模块，属于同一执行流程的组成部分。
2. **调用机制差异** 中断调用：硬件中断通过外部设备信号触发，软件中断通过 INT n 指令引发，整个过程需要经过中断请求、优先级判优、向量表查找（中断号×4）才能获得入口地址。子程序调用：完全由 CALL 指令直接发起，入口地址由指令中的操作数明确指定，不存在中断判优和向量表查询过程。
3. **执行上下文处理** 中断服务程序：CPU 在进入 ISR 时会自动将 FLAGS、CS、IP 压入堆栈，程序员必须手动保存其他可能被修改的寄存器，返回时使用 IRET 指令自动恢复现场。普通子程序：仅需程序员根据需要保存必要寄存器，返回时使用 RET 指令，仅恢复 IP（或 CS:IP）。
4. **设计约束条件** 中断服务程序：执行时间必须尽量短，以避免丢失后续中断请求，同时需要考虑可重入性和中断嵌套问题，通常由系统级开发者编写。普通子程序：执行时间相对宽松，调用关系明确，由应用程序开发者编写，设计自由度较高。
5. **典型应用场景** 中断服务程序：主要用于硬件设备驱动（如键盘、定时器）、系统异常处理（如除零、页故障）和 BIOS/DOS 功能调用。普通子程序：主要用于代码模块化、算法实现以及程序逻辑的结构化分解。

正确区分中断服务子程序与普通子程序的差异，对于编写稳定可靠的汇编程序具有重要意义。在本次实验中，我通过自定义 INT 86H 实现了中断调用，这不仅加深了对中断向量表和 IRET 返回机制的理解，也让我意识到中断处理的实时性和现场保护是保证程序稳定运行的关键。在后续保护模式编程中，中断处理还会涉及更多复杂机制，如中断描述符表（IDT）和特权级切换等。

REP MOVSB 与普通 MOV 指令的区别可以从以下几个方面进行深入分析：

1. **执行方式** REP MOVSB 是带重复前缀的字符串操作指令，一次执行可自动循环多次。普通 MOV 指令是单次数据传送指令，每次只能传送一个字节或字。
2. **操作对象** REP MOVSB 专门针对内存中的字符串块操作，自动使用 SI（源）和

DI（目的）作为地址指针。普通 MOV 指令可操作寄存器、立即数或单个内存单元，不具备字符串批量处理能力。

3. **自动管理机制** REP MOVSB 在每次传送后会自动修改 SI 和 DI（受 DF 方向标志影响），并将 CX 作为计数器自动递减至 0。普通 MOV 指令不会自动修改地址指针和计数器，需要程序员手动编写循环。
4. **性能与适用场景** REP MOVSB 适合大块内存复制，效率极高，是字符串操作的标准指令。普通 MOV 指令适用于单个数据传送或需要精细控制的场合。
5. **方向与标志影响** REP MOVSB 受 CLD（正向）和 STD（反向）指令影响。普通 MOV 指令不受方向标志影响。

掌握 REP MOVSB 与普通 MOV 的区别是实现高效字符串操作的关键，在本次实验的字符串复制中，正确使用 REP MOVSB 显著提高了代码简洁性和执行效率。

段寄存器管理与初始化的重要性可以从以下几个方面进行深入分析：

1. **段寄存器作用** DS 用于数据段访问，ES 用于附加段访问，SS 用于栈段访问，CS 用于代码段访问。每个段寄存器决定了当前指令可访问的内存区域。
2. **初始化必要性** 汇编程序启动时 DS、ES、SS 通常未指向正确段，必须通过 MOV AX,段名 → MOV 段寄存器,AX 进行显式初始化。
3. **常见错误场景** 未正确初始化 DS 或 ES 会导致字符串显示失败、数据访问错误或程序崩溃。
4. **段切换技巧** 当源和目标字符串位于不同段时，需要切换 DS 或 ES，并使用段超越前缀（如 ES:）。
5. **与保护模式的区别** 实模式下段寄存器直接存放段基址，保护模式下变为段选择子，管理机制更加复杂。

正确理解和初始化段寄存器是汇编程序能够正常运行的基础，本次实验中多次出现的显示失败问题均与段寄存器管理不当直接相关。